

DEIN LEBEN.

DEINE ZUKUNFT.

DEIN STUDIUM.

Konzeption und Mastering einer Website mit Basics in HTML5 und CSS3

Studienskript

DMS-KMW-10

Herausgeber:
Prof. Online Akademie für Sales und Marketing
Barmbichl 12
D-83349 Palling

www.smarketing-akademie.de
info@smarketing-akademie.de

DMS-KMW-10
Versions-Nr: 1.0-07.23

© Prof. Online Akademie für Sales und Marketing

Dieses Studienskript ist urheberrechtlich geschützt.

Die Reproduktion in jeglicher Form und/oder die Verarbeitung, Vervielfältigung oder Verbreitung durch elektronische Systeme ist/sind ohne vorherige schriftliche Genehmigung des Herausgebers nicht gestattet.

Sofern Bezeichnungen von Personen nur in weiblicher oder männlicher Form verwendet werden, schließt dies das jeweils andere Geschlecht, ebenso divers, nicht aus.

Alle im Studienskript genannten Marken gehören ihren jeweiligen Eigentümern, auch, wenn Marken und deren Inhaber nicht direkt als solche gekennzeichnet sind.

Die Inhalte dieses Studienskripts wurden mit äußerster Sorgfalt erstellt. Dennoch erheben der Autor und Herausgeber keinen Anspruch auf Vollständigkeit und Richtigkeit und übernimmt keine juristische Verantwortung und keine daraus folgende oder sonstige Haftung, die auf irgendeine Art aus der Benutzung dieser Inhalte – oder Auszügen davon – entstehen.

Alle Rechte vorbehalten.

Inhaltsverzeichnis

1. Begriff der Website, Auszeichnungs- und Programmiersprachen	4
2. Grundlagen CSS3	6
9.1 Hierarchie von CSS-Notationen	9
9.2 Farb- und Größenangaben, Festlegen von Schriften	mittels CSS 11

Übergeordnete Lernziele

Eine Website bildet vor allem im E-Commerce, also dem elektronisch gestützten Handel über das Internet, die Basis für das unternehmerische Agieren in dem Markt, in dem sie sich befindet. Sie stellt Usability- und User-Experience-optimierte Informationen auf hierarchisch angeordneten Seiten zur Verfügung, orientiert sich an der konsequenten Einhaltung des Sales Funnels zur Generierung von Leads und/oder Sales und berücksichtigt darüber hinaus wichtige Faktoren der Suchmaschinenoptimierung. Ihre Gestaltung und Erstellung bzw. Programmierung orientiert sich an den Schritten Planung und Konzeption, Design und operative Umsetzung. Insbesondere für Letztere sind Grundkenntnisse in HTML(5) und CSS(3) erforderlich.

Im Kurs „**Konzeption und Mastering einer Website mit Basics in HTML5 und CSS3**“ beschäftigen wir uns mit den einzelnen Planungs-, Design- und Umsetzungsprozessen sowohl für die Neukonzeption einer Website als auch für deren Relaunch. Dazu evaluieren wir u.a. die Ziele und Zielgruppe(n) einer Website und erfassen wichtige Aspekte entlang des User Interface Designs. Abschließend widmen wir uns der Erstellung/Programmierung einer Website anhand der Grundlagen in HTML5 und CSS3.

Rechtliche Grundlagen, technische Optimierungsmaßnahmen sowie Informationen zu Wartung und Datensicherung runden das Studienskript ab.

Wir wünschen Ihnen viel Freude und Erfolg beim Durcharbeiten des Studienskripts!



1. Begriff der Website, Auszeichnungs- und Programmiersprachen

In diesem Kapitel werden die gebräuchlichsten Bezeichnungen für eine Website, ihre Struktur und ihr Zweck kurz erläutert.

In der Praxis gibt es mehrere Bezeichnungen für die Online-Darstellung eines Unternehmens, die im gleichen Zusammenhang meist synonym verwendet werden. Im weiteren Verlauf dieses Studienskripts werden die Begriffe „Website“ und „Webauftritt“ stellvertretend für eine vollständige „Webseite“, „Homepage“ und „Internetseite“ verwendet.

Eine Website stellt im Sinne des Wissenstransfers in diesem Studienskript ein vollständiges Online-Projekt dar, bestehend aus einer Hauptseite oder auch „Mainpage“ sowie mehreren untereinander verlinkten und strukturell (hierarchisch) abgebildeten Unterseiten (Subpages), die über eine entsprechende Navigation und über unterschiedliche URLs erreichbar sind und zusammen ein einheitliches Design verwenden, das dem Corporate Design eines Unternehmens entspricht. Sie enthält bzw. repräsentiert unternehmensspezifische Inhalte, die im Detail aus Überschriften (Headlines), Informationstexten, Grafiken, Videos, Audioelementen, Tabellen und ggf. dynamischen Inhalten bestehen.

Darüber hinaus kann eine Website spezielle Skripte enthalten, die z.B. über die Programmiersprachen JavaScript, Perl oder PHP abgebildet werden, um bestimmte Operationen ausführen zu können, wie z.B. das Ausfüllen und Absenden von Formularen, das An- und Abmelden bei einem Benutzerkonto oder das Speichern und Abrufen von Informationen in und aus einer angeschlossenen Datenbank.

Die einzelnen Unterseiten eines Webauftritts lassen sich sowohl statisch über spezifische URL-Endungen wie **seite1.html**, **seite1.htm** oder nur über ihren definierten Namen, z.B. **/kategorie** darstellen, oder werden für bestimmte Zwecke dynamisch über ein serverseitiges Skript generiert, ohne dass dies der Endnutzer merklich erkennt. Zu Letzterem zählen beispielsweise Aktionen wie das Hinzufügen von Produkten zu einem Warenkorb, das Auslesen und Einbinden von Daten aus einer Datenbank oder die Darstellung von Suchergebnissen, wobei hierfür i.d.R. entsprechende Parameter an die URL des Abrufs angehängt werden, z.B. **/kategorie?s=suchbegriff**.

Eine Abweichung von der klassischen Website stellen sogenannte „Webportale“ wie z.B. web.de oder gmx.de dar, die nicht nur eine große Menge an Inhalten zu den unterschiedlichsten Themen(-bereichen) enthalten, sondern selbst auf zahlreiche externe Websites verlinken und weiterhin externe Inhalte von Drittwebsites integrieren und ihren Nutzern konsolidiert zur Verfügung stellen.

Grundsätzlich besteht jede Website aus einem HTML-Grundgerüst, wobei HTML (Hypertext Markup Language) eine reine Auszeichnungssprache (Markup Language) ist. Sie ist bis auf wenige Ausnahmen nur in der Lage, rein statische Inhalte auszuliefern.

Für komplexere Anforderungen, z.B. in Verbindung mit einer Datenbank, für Berechnungsaufgaben, für die Verwendung von Formularen oder anderen interaktionsbedürftigen Elementen, ist der Einsatz einer oder mehrerer Programmiersprachen erforderlich, die wiederum in den Quelltext des HTML-Gerüsts einer Website integriert werden.

Dieses Basisgerüst werden wir in **Kapitel 8.2** näher betrachten.

Aufgaben zur Selbstkontrolle ☒

1. Nennen Sie min. fünf Elemente, aus die eine Website bestehen kann.
2. Eine auf HTML-basierende Website kann Funktionen, die auf PHP basieren, aufnehmen und über einen Browser ausführen:
☐ Richtig
☐ Falsch
3. Was ist der Unterschied zwischen einer regulären Website und einem Webportal?



Wissenskontrolle:
Überprüfen Sie, ob
Sie diese Lektion ver-
standen haben.
Die Lösungen dazu
finden Sie hier

Zusammenfassung

Eine Website besteht in der Regel aus miteinander verknüpften und strukturell (hierarchisch) dargestellten Unterseiten, die über eine entsprechende Navigation und unterschiedliche URLs erreichbar sind. Das Design spiegelt i.d.R. das vorherrschende Corporate Design wider. Darüber hinaus besteht eine Website aus inhaltlichen Komponenten wie dem Content und ggf. Skripten, die mittels einer oder mehrerer Programmiersprachen erstellt und eingebettet werden. Das Grundgerüst besteht aus einem HTML-Grundgerüst, wobei HTML (Hypertext Markup Language) eine reine Auszeichnungssprache (Markup Language) darstellt.



Zusammenfassung
dieses Kapitels



Audio-Ausgabe im On-
line-Campus verfüg-
bar

2. Grundlagen CSS3

In diesem Kapitel befassen wir uns mit den wichtigsten CSS-Elementen, um grafische Änderungen einer Website vornehmen zu können.

“CSS” (Cascading Style Sheets) ist als Gestaltungs- und Formatierungssprache für das Design eines HTML-Dokuments bzw. von Elementen zuständig. Gegenüber der Vorgängerversion CSS ist es mit CSS3 möglich responsive Designs, die Einbindung von Animationen und Texteffekten sowie zusätzliche Funktionen, wie u.a. von Pseudoklassen, umzusetzen. Im weiteren Verlauf dieses Studienskripts werden wir an Stelle des Begriffs CSS3 der Einfachheit halber „CSS“ verwenden.

Die CSS-Styles einer Website bzw. von spezifischen Elementen lassen sich in Browsern (Laptops und Desktop-PCs) durch einen Rechtsklick und Klick auf „Element untersuchen“ o.ä. im rechten Seitenbereich aufrufen. In diesem Bereich können weiterhin die verschiedenen Eigenschaften in Echtzeit geändert werden, wobei Änderungen sofort sichtbar sind, jedoch nicht gespeichert werden. Auf diese Weise können Sie selbst Übungen bzw. Tests an verschiedenen Elementen durchführen und so das Gelernte in Auszügen direkt anwenden.

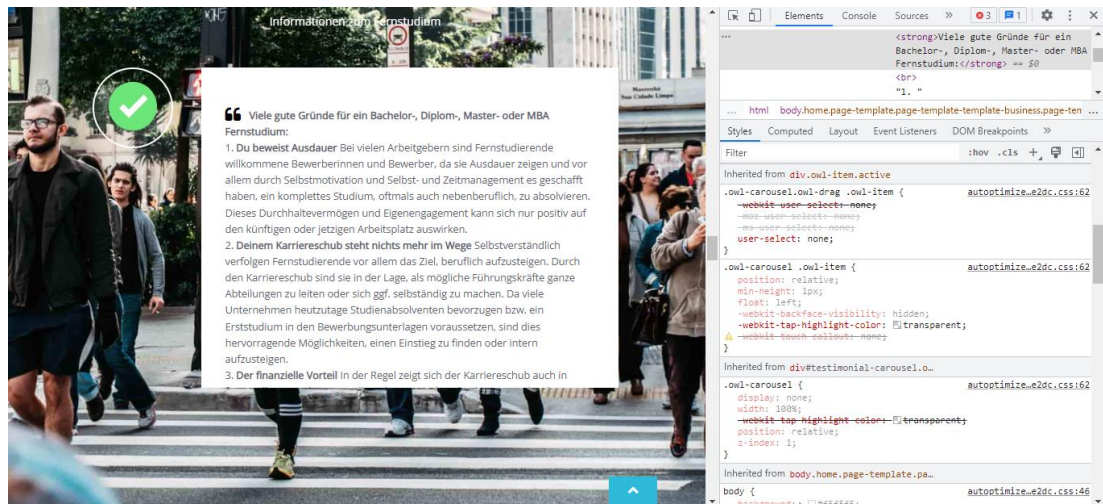


Abbildung 1: Aufruf der CSS-Styles einer Website über den Browser

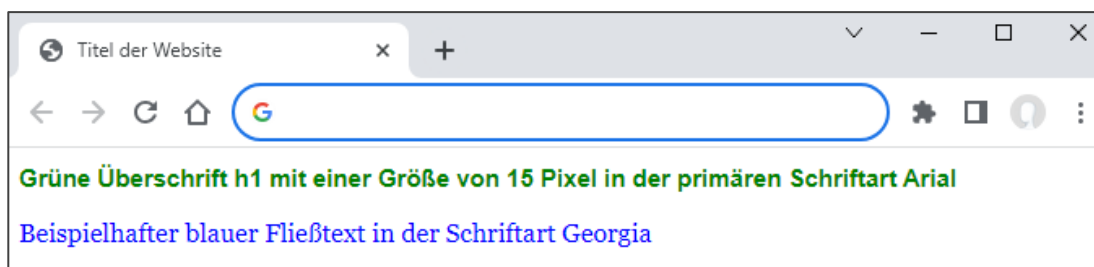
Es gibt drei Methoden der Integration von CSS-Angaben in eine Website:

- Notation im `<head></head>`-Bereich
- Inline für solche Blockelemente, die eine Zuweisung von CSS-Angaben erlauben
- als ausgegliederte separate CSS-Datei (Dateiendung .css), die per Verlinkung eingebunden wird

Die Notation im `<head></head>`-Bereich erfolgt anhand folgendem Beispiel, wobei wir bereits erste Styles kennenlernen werden:

```
<head>
<style type="text/css"> ←einleitendes style-Tag mit Angabe des Typs. Achtung: mit HTML5
kann auf die type-Angabe verzichtet werden. Die Notation wäre dann schlicht <style>
h1 {font-size:15px; ←Zuordnung einer Schriftgröße mittels font-size von 15 Pixel für h1-Über-
schriften
font-family:Arial,sans-serif; ←Zuordnung der Schriftart (font-family) Arial zu h1 und No-
tation der Fallback-Schriftart sans-serif
color:green; ←Zuordnung der Schriftfarbe (color) grün zu h1
}
p,div,li,th,td {←Nennung mehrerer durch Komma und Leerzeichen getrennter Elemente, für
welche die nachfolgenden Styles gelten sollen
font-family:Georgia,Arial,sans-serif; ←Nennung zweier hierarchischer Fallback-
Schriftarten
color:blue;
}
</style> ←abschließendes style-Tag
</head>
<body>
<h1>Grüne Überschrift h1 mit einer Größe von 15 Pixel in der primären
Schriftart Arial</h1>
<p>Beispielhafter blauer Fließtext in der Schriftart Georgia</p>
</body>
```

Ausgabe im Browser:



Man beachte die Art und Weise der Notation von CSS-Angaben innerhalb von `<style></style>`: Zunächst wird das relevante Element referenziert, z.B. **h1**. Anschließend werden innerhalb einer einleitenden und abschließenden geschwungenen Klammer (`{` und `}`) ein oder mehrere Attribute mit Doppelpunkt (`:`), z.B. **font-size:** und dessen Wertzuweisung, z.B. **15px** notiert, wobei Letztere mit einem Semikolon (`;`) abgeschlossen wird. Das abschließende Semikolon ist nur dann verpflichtend, sofern eine weitere Wertzuweisung folgt. Im Laufe dieses Studienskripts werden wir allerdings zur besseren Darstellung das Semikolon stets auch bei Einzelanweisungen anwenden.

Somit lautet, lt. W3-Konsortium, die Notation einer einzelnen CSS-Zuweisung im `<head></head>`-Bereich sowie auch in einer `.css`-Datei auf

Element {Eigenschaft:Wert;}

sowie bei mehreren Zuweisungen für ein einzelnes Element auf

Element {Eigenschaft1:Wert;Eigenschaft2:Wert;Eigenschaft3:Wert;} etc.

In unserem vorherigen Beispiel geben wir für alle **h1**-Überschriften sowie für die Elemente **p**, **div**, **li**, **th** und **td** des Dokuments global gültige Werte vor, sofern diese nicht explizit durch Inline-Styles überschrieben werden.

Sofern für bestimmte Elemente kein Style zugewiesen wurde, stellen Browser diese in einer standardisierten Darstellung dar.

Eine zweite Variante der Integration von CSS-Styles in ein Dokument ist das Einlesen einer ausgelagerten .css-Datei im **<head></head>**-Bereich, die somit für dieses Dokument global gültig sind, z.B.

```
<link rel="stylesheet" type="text/css" href="style.css">
```

oder alternativ über die @import-Regel:

```
<style type="text/css">  
@import url("style.css") all; p,li {  
font-size:10px;  
}  
</style>
```

Im letzten Beispiel legen wir mit der Angabe **all** fest, dass sämtliche Styles der .css-Datei importiert werden sollen, jedoch mit der Ausnahme, dass explizit für dieses Dokument die Elemente **p** und **li** eine andere Schriftgröße, hier 10 Pixel, erhalten sollen, als wie ursprünglich in der .css-Datei festgelegt.

Generell benötigt eine separat erstellte .css-Datei keine spezifischen Kopfangaben, d.h. entsprechende Notationen werden direkt in diese eintragen, z.B. mittels eines Editors:

```
h1 {font-size:15px;}  
p {font-family:Arial,sans-serif;}
```

etc.

Das Nutzen einer externen CSS-Datei ist dann relevant, sofern sämtliche Seiten eines Webauftritts einheitliche Wertzuweisungen erhalten sollen. Styles, die im **<head></head>**-Bereich platziert werden, sind für Unterseiten relevant, die jeweils unterschiedliche Styles beinhalten sollen.

Die letzte Variante stellt die Notation von Inline-CSS dar, die einem Element direkt zugeordnet werden kann. Laut des W3-Konsortiums lautet die Notation von Inline-CSS auf

```
style="Eigenschaft1:Wert;Eigenschaft2:Wert;"
```

z.B.

`<p style="color:red;">Das ist ein roter Text</p>` ←color (Farbe) für Textfarbe

oder für die Notation mehrerer Styles:

`<body style="background-image: url('beispielgrafik.jpg');font-family:Arial,Helvetica,sans-serif;">` ←Notation eines Hintergrundbildes sowie einer Schriftart für den gesamten body-Bereich. Man beachte bei beispielgrafik.jpg die unterschiedlichen Anführungsstriche. Innerhalb von "" müssen daher die Anführungsstriche ' ' verwendet werden

Ausgabe im Browser:



Das Prinzip der Vererbung besagt, dass ein Element die Style-Eigenschaften des übergeordneten Elements erbt, sofern für dieses kein spezifischer eigener Style zugeordnet wurde. So erbt bei folgendem Beispiel einer Inline-Zuweisung das Element `<span>` die übergeordneten Eigenschaften von `p`:

`<p style="color:red;">Die ist immer noch ein <span>roter Text</span></p>`

Im weiteren Verlauf dieses Studienskripts verwenden wir überwiegend Inline-Styles, wobei sich ihre Notationen auch über entsprechende Formulierungen im Kopfbereich oder über eine .css-Datei verwenden ließen.

9.1 Hierarchie von CSS-Notationen

Wie wir bereits kennengelernt haben, können CSS-Styles sowohl im `<head></head>`-Bereich, in diesem über das Einlesen einer verlinkten .css-Datei und Inline definiert werden. Dabei ist es möglich, alle diese drei Varianten für eine Webseite anzuwenden.

Nun wäre die Frage zu klären, welche dieser Notationen bei gleichzeitiger Anwendung „Vorrang“ vor anderen hat.

Dazu gelten folgende einfache Regeln:

- Style-Angaben im `<style></style>`-Bereich des Dokumentenkopfes haben Vorrang vor extern eingebundenen CSS-Dateien
- Inline-Styles haben Vorrang vor sämtlichen Notationen im Kopfbereich

Hieraus ergibt sich eine klare Hierarchie: Verlinkte CSS-Styles erhalten nur dann volle Gültigkeit, sofern keine weiteren „überschreibenden“ Styles im Kopfbereich oder Inline definiert wurden. Dieselbe Hierarchie gilt für Styles im Kopfbereich und Inline, sofern keine verlinkte CSS-Datei existiert.

Inline-Styles haben stets für das jeweilige Element Vorrang, für welche sie notiert werden. Hierbei gilt zu beachten, dass diese Logik natürlich nur für solche Elemente gültig ist, für die mehrere Style-Zuordnungen zutreffen. Definiert eine verlinkte CSS-Datei beispielsweise Style-Angaben zu einer **h1**-Überschrift, die weder im Kopfbereich noch als Inline zugewiesene Werte enthält, dann behält diese Überschrift für dieses Dokument ihre festgelegten Eigenschaften aus der verlinkten Datei. Würden wir hingegen für diese **h1**-Überschrift eine zusätzliche überschreibende Inline-Wertzuzuweisung definieren, erhält diese den Wert aus der Inline-Notation.

Ein Beispiel:

Eine verlinkte .css-Datei enthält eine Wertzuweisung für die Schriftgröße einer **h1**-Überschrift von 20 Pixel (**font-size: 20px;**), die für dieses Dokument global gültig ist. Über eine Inline-Notation weisen wir nun einer spezifischen **h1**-Überschrift den Wert von 30 Pixel zu. Der Output würde nun für diese spezifische **h1**-Überschrift eine Schriftgröße von 30 Pixel darstellen.

Oftmals ist es notwendig bestimmten Elementen explizit keinen Wert zuzuweisen und somit ihren Output zu unterdrücken. Dazu übergeben wir den Wert **none**, z.B. **border:none;** (hier die Unterdrückung eines Rahmens). Mit **none** werden Elemente schlichtweg nicht erzeugt. Über die Notation **visibility:hidden;** wird ein Element vollständig ausgeblendet.

Weiterhin ist es häufig erforderlich, bestimmte Wertzuweisungen für ein Element zu erzwingen, sofern sich im Laufe der Programmierung inkonsistente Zuweisungen ergeben haben. Eine solche erzwungene Wertzuweisung wird über ein nachgestelltes **!important** referenziert, z.B. **border:1px solid blue!important;** ←erzwungener blauer Rahmen, mit einer Dicke von 1 Pixel und einer durchgehenden Umrandung

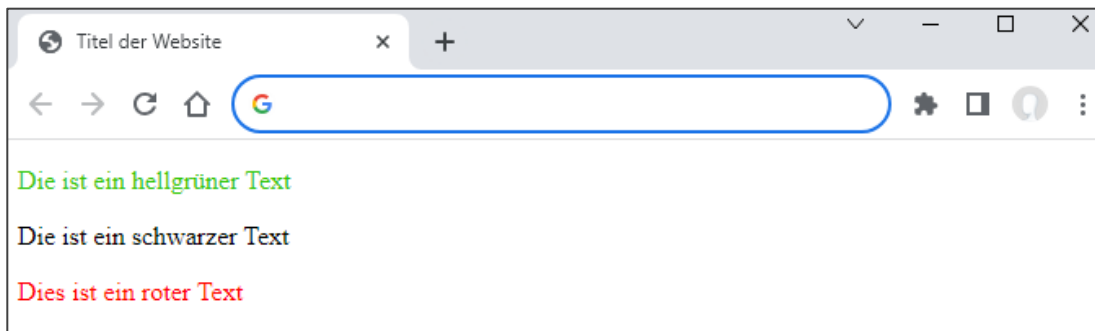
Hierbei erkennt der Browser, dass diese Zuweisung in jedem Falle Vorrang vor allen anderen übergeordneten Zuweisungen haben soll.

9.2 Farb- und Größenangaben, Festlegen von Schriften mittels CSS

Die Farbgestaltung per CSS erfolgt entweder über

- den RGB-Farbraum (Rot, Grün, Blau), mit jeweils Werten zwischen 0 und 255. Die Farbgebung 0, 0, 0 ergäbe den Farbton schwarz, 50, 200, 10 hingegen die Farbe hellgrün. Die Notation erfolgt über `<p style="color:rgb(50,200,10);">Die ist ein hellgrüner Text</p>`
- sechs Hexadezimalziffern (0-9) sowie mit den Buchstaben A, B, C, D, E und F. Die Ziffern 1 und 2 stellen dabei den Rotanteil, die Ziffern 3 und 4 den Grünanteil und die Ziffern 5 und 6 den Blauanteil dar. Die Notation erfolgt über `<p style="color:#000000;">Die ist ein schwarzer Text</p>`. Speziell diese Farbkombination ließe sich auch in abgekürzter Form als `#000` notieren. Hierbei erfolgt eine Verdoppelung der Ziffernfolge. An einem weiteren Beispiel ließe sich anstelle von `#8888AA` nur `#88A` für einen Grauton notieren
- eine simple direkte Farbbezeichnung mit englischen Begriffen, z.B. `<p style="color:red;">Dies ist ein roter Text</p>`

Ausgabe im Browser:

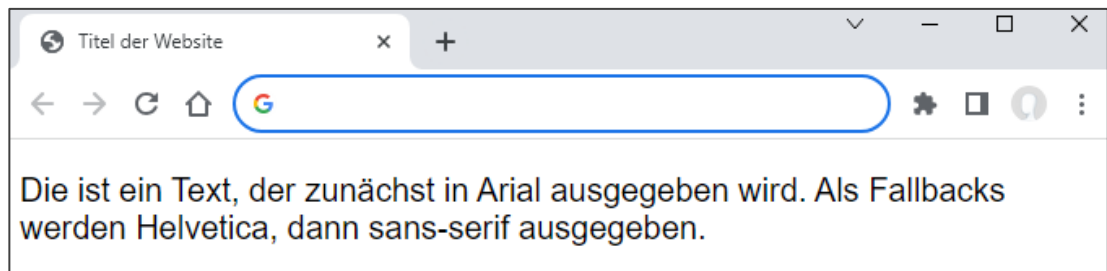


Größenangaben lassen sich mittels CSS mit folgenden Angabewerten zuordnen:

- Pixel, z.B. `width:200px;` für eine Elementbreite von 200 Pixel
- Zentimeter, z.B. `height:0.8cm;` für eine Elementhöhe von 0,8 cm. Angaben in Zentimeter sind vornehmlich für die Ausgabe auf Druckern geeignet
- prozentualer Anteil, z.B. `font-size:0.5em;` für eine Schriftgröße von 50%, gemessen am übergeordneten (Eltern-)Element
- prozentuale Größe, z.B. `min-width:60%;` für ein Element, das eine Mindestbreite von 60% von der verfügbaren Breite des Dokuments einnehmen soll

Die Notation von Schriftarten erfolgt beispielsweise über `<p style="font-family:Arial,Helvetica,sans-serif;font-size:20px;">Die ist ein Text, der zunächst in Arial ausgegeben wird. Als Fallbacks werden Helvetica, dann sans-serif ausgegeben.</p>`

Ausgabe im Browser:



Da nicht jede Schriftart automatisch auf den Ausgabegeräten angezeigt wird, insbesondere bei „exotischeren“ Schriftarten, werden zusätzlich Fallback-Schriftarten definiert - im obigen Beispiel **Helvetica** sowie **sans-serif** für eine serifenlose Schriftart.

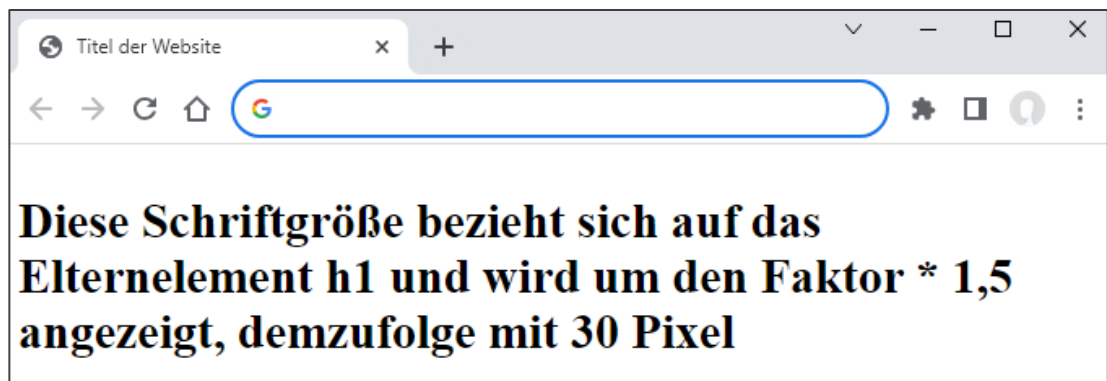
Die Notation einer eigenen auf dem Server hinterlegten Schriftart wird mit **@font-face** gebildet, z.B. **@font-face {font-family:Name der Schriftart;src:url("pfad zur Schriftdatei/schrift.ttf");}** (i)

Neben Pixelangaben lassen sich Schriftgrößen zudem über einen **em**-Wert definieren. Dieser orientiert sich am übergeordneten (Eltern-)Element und berechnet diesen über eine entsprechende Wertzuweisung für das betreffende Element neu.

Ein Beispiel:

```
<h1 id="Elternelement-h1" style="font-size:20px;">
<p style="font-size:1.5em;">Diese Schriftgröße bezieht sich auf das El-
ternelement h1 und wird um den Faktor * 1,5 angezeigt, demzufolge mit
30 Pixel</p>
</h1>
```

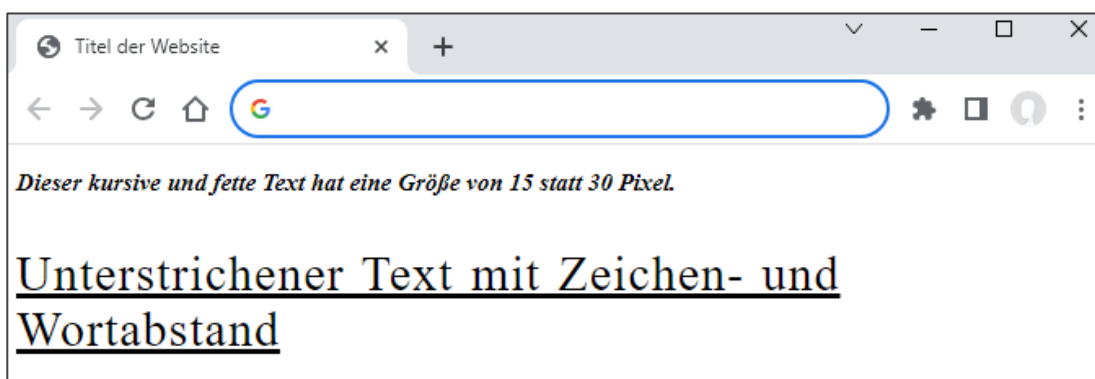
Ausgabe im Browser:



Ein weiteres Beispiel einer um die Hälfte (**0.5em**) reduzierten Schriftgröße, welches sich somit auf Basis des übergeordneten Elements body neu berechnet:

```
<body style="font-size:30px;">
<p style="font-size:0.5em;font-weight:bold;font-style:italic;">Dieser
kursive und fette Text hat eine Größe von 15 statt 30 Pixel.</p> ←font-
weight für die Stärke der Schrift, hier bold für „fett“, font-style: italic für eine kursive Darstellung
<p style="letter-spacing:1px;word-spacing:2px;text-decoration:under-
line;">Unterstrichener Text mit Zeichen- und Wortabstand</p> ←letter-spa-
cing für den Abstand der Buchstaben, word-spacing für den Abstand der Wörter, text-decoration für eine
Unterstreichung
</body>
```

Ausgabe im Browser:



Im letzten Beispiel haben wir verschiedene Schriftformatierungen festgelegt: Per **font-weight** die Schriftstärke (**bold** = fett, **normal**, **bolder**=fetter, **lighter**=dünner oder numerisch **100** (sehr dünn) bis **900** (sehr fett) sowie **inherit** (Schriftstärke des Elternelements)), eine kursive Darstellung mittels **font style** (**italic** oder **oblique**), mittels **letter-spacing** den Zeichenabstand, per **word-spacing** den Abstand zwischen Wörtern und mit **text-decoration** eine weitere Gestaltung, hier **underline** für unterstrichen.

Ende der Vorschau

```
78 // Trim the path and replace backslashes with forward slashes
79 $image_src = trim(preg_replace('/\\\\\\\\/', '/', $image_src), '/');
80 $SESSION['_CAPTCHA']['config'] = serialize($captcha_config);
81 return array(
82     'code' => $captcha_config['code'],
83     'image_src' => $image_src
84 );
85 }
86
87
88 if ( !function_exists('hex2rgb') ) {
89     function hex2rgb($hex_str, $return_string = false, $separator = ',') {
90         $hex_str = preg_replace("/^[^0-9A-Fa-f]/", '', $hex_str); // Gets a proper hex string
91         $rgb_array = array();
92         if ( strlen($hex_str) == 6 ) {
93             $color_val = hexdec($hex_str);
94             $rgb_array['r'] = 0xFF & ($color_val >> 0x10);
95             $rgb_array['g'] = 0xFF & ($color_val >> 0x8);
96             $rgb_array['b'] = 0xFF & $color_val;
97         } elseif ( strlen($hex_str) == 3 ) {
98             $rgb_array['r'] = hexdec(str_repeat(substr($hex_str, 0, 1), 2));
99             $rgb_array['g'] = hexdec(str_repeat(substr($hex_str, 1, 1), 2));
100             $rgb_array['b'] = hexdec(str_repeat(substr($hex_str, 2, 1), 2));
101         } else {
102             return false;
103         }
104         return $return_string ? implode($separator, $rgb_array) : $rgb_array;
105     }
106 }
107
108 // Draw the image
109 if ( isset($_GET['code']) ) {
110     // ...
111 }
```